AI BUILD LAB. COHORT 1. MAY 3, 2026

You're about to install an Agent Native OS in one day.

110 students. Two tracks. One capstone. Six hours.

By 5pm: your own Agent Native OS, installed, personalized, and running. The morning brief is one expression of what it can do. The repo has hundreds more.

OPENING

Who’s in the room

4D Track

- First time in Claude Code, or early experience
- You’re here to see what’s possible
- Leave with a working foundation

8D Track

- You live in the terminal. Claude Code is a daily or regular tool
- You have confidence with it, even if you don’t know everything
- You want to go deeper, not just get it running

Default is 4D. If any 8D criteria is ”no,” 4D is the right call. You can switch any time. No questions asked.

If you start in 4D today, you’re welcome back. Lab members can retake this workshop as many times as they want before Summer Camp opens in July. Our goal: you should feel **8D-comfortable** by the time Summer Camp starts. Today is the first pass, not the only pass.

OPENING

What you bought

"You did not buy the morning brief. You bought the architecture."

The morning brief is one expression of what this architecture can do. The repo has hundreds more, and your Claude knows how to install all of them.

YOU LEAVE TODAY WITH

- An Agent-Native OS, installed, personalized, running
- MCPs wired, slash commands authored, a scheduled loop
- 7,300-transcript Cairns vault with semantic search and graph relationships
- 12+ blueprints ready to deploy, morning brief is one
- Documentation for every tool in the stack, indexed and searchable by Claude
- A school that keeps teaching you after today

TYLER'S LIVE FLEET, WHAT THIS COMPOUNDS TO

Transcripts captured daily	56,621
Skills in daily fleet	92
MCPs / tools wired	23 / 390+
Automations running daily	24
Heartbeat runs per month	8,640, free
Monthly fixed spend	$800 total

Repo = school. Claude = teacher. We are the TAs in the hallway.

OPENING

What this is actually worth

You paid **$497**. Here's what the pieces would cost if you commissioned them.

Pre-built repo 14,000 lines, 11 guides, 8 production slash commands, 3 calibrated track exercises, 17K-word use-case library, templates	$80K–$200K
Cairns architecture access Tyler's 18 months of design + production system (88K Neo4j nodes)	$20K–$50K
7 hours live with 4 senior instructors	$3K–$8K
Working Morning Brief deployed by 5pm Vercel + Supabase + Telegram, custom dev equivalent	$5K–$10K
Recording + Slack + Lab pathway	$2K–$5K
Honest range	**$110K–$270K**

WHY IT'S WORTH THIS MUCH

- **It's been run.** Tyler operates ~92 slash commands daily in his actual business. The ones in your repo are battle-tested, not theoretical.
- **Cairns is unique IP.** Nobody else is teaching this architecture. Consultancies that could would charge $50K+ for the architecture alone.
- **Time-to-value.** A consulting engagement to build this from scratch would take 4–8 months. You walk out at 5pm with it running.
- **It evolves.** The repo updates as the field updates. Lab membership = ongoing access to Tyler's working tools.

> **$497 → $110K minimum.**
> That's a **220x value multiple.** If you only get the repo and nothing else, you've already won.

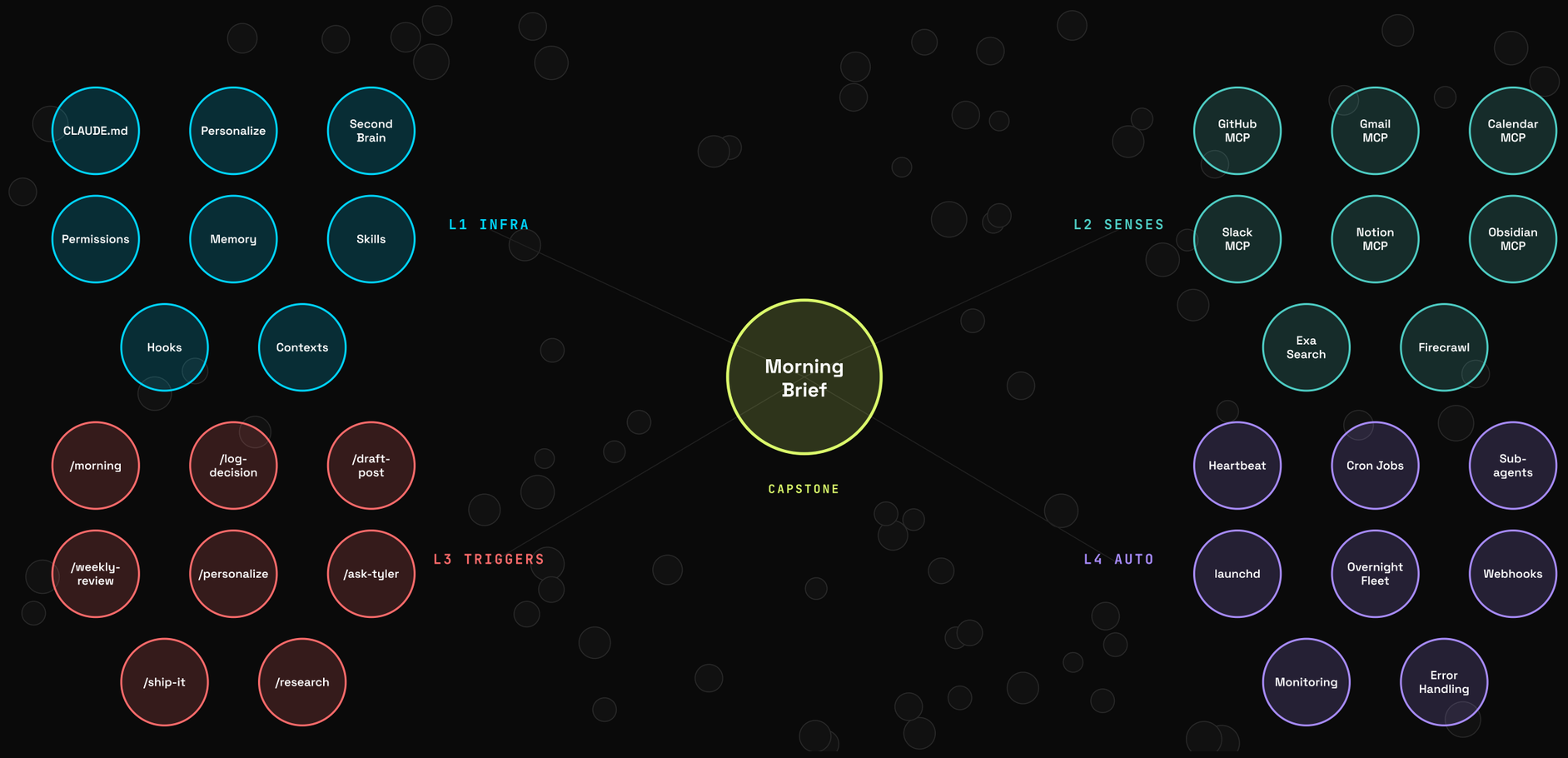
OPENING

Today's nodes, 50 of 250

aibuildlab.com/ecosystem

L1 Infrastructure · L2 Senses · L3 Triggers · L4 Autonomy

dim = real, not covered today



Dim nodes are real capabilities in the repo. At 4:30 PM Tyler toggles to all 250. **"You bought THIS."**

OPENING

The repo is the school.

`~/GitHub/agent-native-os/`

Everything you install today lives here. Everything you build next lives here. Claude reads it, teaches from it, and extends it.

WHAT IS IN IT

- **Cairns AI vault**, 7,300 AI video transcripts with semantic search and graph relationships. Three brain transplants for your Claude.
- **Tool documentation**, every tool in the stack, scraped and indexed. Claude teaches from it.
- **Blueprints**, 12+ pre-built agent systems (morning brief, Bland phone agent, Cairns, and more)
- **Guides**, step-by-step walkthroughs for every layer (01 through 09)
- **Commands**, 8 slash commands ready to copy and invoke
- **Templates**. CLAUDE.md template, Obsidian Cairns starter vault
- **Configs**, settings.json starters, MCP config examples

HOW TO USE IT AFTER TODAY

```
cd ~/GitHub/agent-native-os
claude
```

"I just finished the workshop.
Walk me through the next thing."

Your Claude knows where everything is. It reads the guides. It teaches you the next layer. You never have to figure out what to do next.

> **Repo = school. Claude = teacher. We = TAs in the hallway.**

OPENING

What today is NOT

NOT THIS

- ~~"Make AI agents"~~
- ~~A tools tour (not 47 platforms)~~
- ~~A Claude Code tutorial (the docs cover that)~~
- ~~A magic show with no foundation~~

THIS

- **Install agent operations.** The plumbing, the discipline, the kill switch, the audit trail.
- **One node in the repo**, which is a full course you work through with Claude after today.

> **Repo = school.** Claude = teacher.

OPENING

The 6 properties of any agent you'd trust your business to

01

Bounded scope

One sentence: what's the job, what's off-limits.

02

Inspectable state

What did it do, when, with what, at what cost.

03

Resumable execution

Crashes survive. Reboots survive. Ctrl-C survives.

04

Scoreable output

Not "looked good." A diff, a row, a transcript with a rubric.

05

Escalation path

The agent stops and asks. Human in the loop is first-class.

06

Kill switch

Budget, time, retry, and manual stop. Wired before ship, not after.

`6/6 = operational` `3/6 = prototype` `0/6 = liability`

OPENING

Rules of engagement

ON THE TRACKS

- Everyone gets the 8D assets. 4D is earlier on the roadmap, not less.
- 4D: get to 8D material by working with Claude after today. The repo has a full course.
- Today is an **install session**, not the course. The course runs on your machine after today.

HOW TODAY RUNS

Outcomes → Concept → Demo → 4D split / 8D split → Rejoin

Q&A in chat during main segments. Deeper Q&A in breakoff rooms.

LEAVE WITH ALL 6

1. **Claude Max active**
   The engine. Max 5x ($100/mo) minimum.
2. **Claude desktop installed + signed in**
   Primary surface. Also installs the CLI.
3. **Claude Code CLI working**
   The agent that reads the repo and runs your OS.
4. **GitHub CLI authenticated**
   Clone the repo, pull updates.
5. **agent-native-os repo cloned**
   The school. Everything on disk.
6. **/personalize run, CLAUDE.md is yours**
   Your personalization layer. Without it: generic assistant. With it: your agent.

OPENING

Security + permissions, before we build anything

> Giving Claude Code access to your machine feels different from logging into Cassidy or ChatGPT. It IS different, it can read files, run commands, and write to your disk. That is exactly why this layer exists first, before anything else.

TRUST PROMPTS, HOW CLAUDE ASKS PERMISSION

SETTING	WHEN TO USE IT
Allow	Your machine, your files, your repos. Claude can act freely.
Ask	New directory, unfamiliar context, anything you want to watch first.
Deny	Anything that doesn't match the task. Writing task asking for SSH keys? Ask why.

> Guard rail, not a wall. Leave prompts on today. Tyler runs auto mode because he has hooks catching anything destructive. You'll get there.

TERMINAL PASSWORD SILENCE

No dots. No asterisks. No cursor movement. This is intentional, decades-old Unix behavior so bystanders can't read your password length. Type it and press Return. If wrong, it re-prompts.

API KEYS, THE ONE RULE

DO

- **Default: 1Password with `op://` refs** (the recommended path, secrets never sit in plaintext)
- Emergency fallback only: `.env` with `chmod 600 .env`
- Rotate immediately if exposed (also check git history)

DON'T

- Paste a key into any chat. Claude, Slack, anywhere
- Commit `.env` to git
- Share or screenshot a key

WHAT CLAUDE CAN AND CAN'T SEE

- **Can see:** dirs you've permitted, content you paste, connected MCP outputs
- **Can't see:** unpointed files, 1Password items unless explicit, other apps
- Your `.env` stays on your hard drive. Anything you paste into chat is sent to Anthropic to generate the response.
- Anthropic deletes chat history within 30 days of you deleting it.

> **The one rule:** don't put secrets in chat. Covers 90% of the risk.

OPENING

What Claude Code actually is

Most AI you've used lives in a website. You type, it answers, you copy the answer somewhere else.

Claude Code lives on your computer. Same Claude model underneath. But this version has permission to:

- **Read your files**, anywhere on your hard drive you point it
- **Write and edit files**, actually do the work, not describe it
- **Run commands**, install things, push code, send messages
- **Use connected tools**. Gmail, Calendar, GitHub, your notes, anything you wire in

It doesn't just answer questions. **It does the work.** You set the goal and the guardrails. It executes.

> **Three places it lives:** the **Desktop app** (where most of today happens), the **Terminal** via the `claude` command (where the overnight stuff runs), and the **VS Code extension** (when you're editing code side by side). Same brain. Three windows.

OPENING

What makes Claude Code yours

A fresh install is a smart stranger. By 5pm it'll be a specialist that knows you. **Seven pieces, all live in a folder on your computer (the `agent-native-os` repo):**

PIECE	WHAT IT IS	PLAIN VERSION	LAYER
CLAUDE.md	A markdown file Claude reads at the start of every session	Tells Claude who you are, what you're working on, how you like to work	L1
MCPs	Standardized connectors apps build for AI	A polished door an app opens for AI. OAuth-based. (Gmail, Calendar, Notion)	L2
CLIs	Command-line tools Claude runs in your shell	Tools built for humans (`gh`, `op`, `vercel`). Claude uses them by typing commands like a person would.	L2
Slash commands	Shortcuts you type to trigger a workflow	`/morning`, `/log-decision`, one word kicks off a multi-step job you wrote	L3
Skills	Pre-packaged know-how Claude loads when it's relevant	Like an instruction manual Claude pulls off the shelf for a specific task	L3
Subagents	Specialized AI workers Claude spawns to do focused jobs in parallel	Claude delegates to mini-Claudes, one researches, one writes, one reviews. Faster and cleaner.	L3
Schedules	Timers that run Claude jobs without you	Wake up at 6am, run the morning brief, ping you on Telegram	L4

Plus, for power users (in the repo, not on this slide): hooks (scripts that fire on tool events as a safety net), memory (Claude's auto-saved cross-session notes), output styles, status line. **The repo ships starter versions of all of these.** Today you install. All week you edit.

OPENING

Permissions, the safety net

Before Claude does anything with real-world effects, writing a file, running a command, sending a message, **it asks.**

Three rule types:

- **Allow**, pre-approved. Claude just does it. (e.g., read files in your repo, run `git status`)
- **Ask**. Claude pauses and waits for you to confirm. (the default for anything new)
- **Deny**, never, not even with confirmation. (e.g., delete files, touch your password manager)

You set these once in a settings file. Claude respects them every session.

Beginners

Leave the prompts on. Watch what Claude wants to do before you let it.

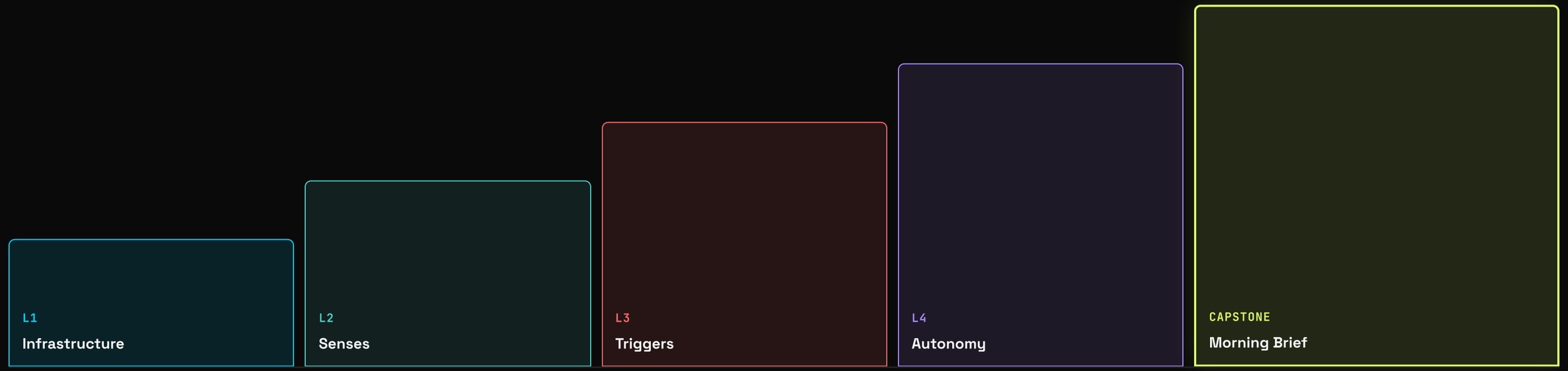
Power users

Pre-approve more. Scope folders tightly. Add hooks to catch anything risky.

Both are correct. You move along that spectrum at your own pace.

OPENING

Four layers. Each one builds on the last.



Your computer learns who you are

Your AI can see your world

Your AI takes action on a single word

Your AI runs while you sleep

You wake up to a brief your AI built overnight

Each layer unlocks the next. All of it runs on the same Claude Code install you set up this morning.

OPENING

What each track leaves with

AREA	4D	8D
L1 Infrastructure	Pre-filled CLAUDE.md, 3 sections. Claude knows your name.	From scratch, 5+ sections. Off-limits rules first. Cairns blueprint after.
L2 Senses	GitHub MCP + Gmail/Calendar connectors. `gh` CLI in active use. One secret in 1Password via `op://`.	3+ MCPs + every secret in 1Password + shell auto-load + one custom Firecrawl→CLI→Skill.
L3 Triggers	One slash command installed and invoked.	Two commands from scratch. `/log-decision` with `$ARGUMENTS` writes to second-brain.
L4 Autonomy	Plan the cron line. Document the kill switch.	Job scheduled live. Kill switch documented. Remote Control connected. Optional Telegram.
Capstone	Vercel deploy + Obsidian markdown output. Telegram ping at 7am. Working URL.	Full Vercel + Supabase. Telegram + Slack + optional iMessage. 2-3 custom sources.

> Both tracks converge at the capstone. Both leave with a working agent.

OPENING

How today works

Four layers. Each follows the same rhythm:

Outcomes table → Concept (together) → Demo (together) → 4D build / 8D build → Rejoin

Four loops. Then everyone merges for the morning brief capstone.

You're not writing prompts from scratch. For every build step, we drop a ready-to-use prompt into **#agent-native-os**. You paste it into your own Claude Code, customize the parts that are about you, and run it. The prompts are the scaffolding. You're the content.

To switch tracks: post in #agent-native-os. No questions asked. Separate Zoom rooms, not breakout rooms, so recordings work.

Q&A in main segments: post in chat. TAs ask it live so everyone hears the answer. Deeper Q&A in breakoff rooms.

OPENING

Today’s schedule

TIME (CT)	WHAT	WHO'S THERE
10:00 – 10:15	**Opening**, what you bought, how today works, security	Everyone together
10:15 – 11:00	**L1 Infrastructure**, personalization, CLAUDE.md	4D Room A · 8D Room B
11:00 – 12:00	**L2 Senses**. MCPs + CLIs, Cairns vault	4D Room A · 8D Room B
12:00 – 1:00	**Lunch + Help Rooms**	4D Help Room · 8D Help Room
1:00 – 2:00	**L3 Triggers**, slash commands	4D Room A · 8D Room B
2:00 – 3:30	**L4 Autonomy**, scheduling, remote control	4D Room A · 8D Room B
3:30 – 4:30	**Capstone**. Morning Brief build	Everyone together
4:30 – 5:00	**Closing**, showcase, what comes next	Everyone together

> All times CT. Room links post in **#agent-native-os** at each split. The lunch break is also catch-up time, nobody falls behind permanently.

LAYER 1

10:15 AM

Personalization.

Your computer learns who you are and how you work.

LAYER 1. CONCEPT

Your computer learns who you are and how you work.

Without personalization: Claude is a generic assistant.

It doesn't know your name, your role, what you're working on, or what you'd never want it to do. Every session starts from zero.

With personalization: Claude loads your context every session.

It knows who you are, what you care about, what your standing rules are, before you type a single word.

> Same model. Same machine. **Different context = different agent.**

HOW IT WORKS

- `~/.claude/CLAUDE.md`, global, loaded every session
- `.claude/CLAUDE.md` at the project root, per-project rules
- Claude reads it automatically. You don't remind it.
- `/personalize` runs a 12-question interview and writes the file for you

> **Think of it like:** a job description you hand a new employee on day one. Without it they make assumptions. With it they know exactly what you care about and what they're never allowed to do.

What you're building in L1

	4D	8D
CLAUDE.md	Run `/personalize`. The agent walks you through ~21 questions across 3 sections (About You / About Your Work / How You Learn) and writes the profile into your `CLAUDE.md`. You review and edit.	Same starting point, run `/personalize` too. Then go further: hierarchical CLAUDE.md cascade (global → project → worktree) + a script that auto-updates your "Current Projects" section.
Second brain	Set up Obsidian with PARA folders (Inbox, Areas, Projects, Resources, Archive). Drop in 3 real notes. Point Claude at it.	Install the `obsidian-cairns-starter` template (adds `daily-briefs`, `decisions`, `meeting-prep`, `cairns`). Read the Cairns 5-doc guide. Build a student-scale mini-Cairns.
Done when	You can ask Claude *"based on my CLAUDE.md, what are the most important things you know about me?"* and the answer matches what you wrote.	Your three-tier CLAUDE.md cascade is loading correctly, and Claude can find a real note in your Cairns starter when asked about a project.

LAYER 1. DEMO

/personalize live

1. Tyler runs `/personalize`
2. Agent walks ~21 questions in 3 sections (About You / About Your Work / How You Learn). Conversational, in batches of 3-5.
3. Agent writes the profile into the project's `CLAUDE.md`
4. Tyler asks: *"what's on my plate today"*
5. Same machine. Two minutes later. Completely different agent.

> Watch for this: Claude addresses Tyler by name without being asked. Same model. Different context. Different agent.

> **8D extension:** Tyler shows three CLAUDE.md files cascading, global, project, worktree. Opens `settings.json`, ~90 auto-allow commands. Asks "what do you know about me?", answer reads like a personnel file.

LAYER 1. BUILD

Now you build it

4D Track

- Run `/personalize` in your Claude Code
- Answer the ~21 questions honestly (3 sections, batches of 3-5)
- Review the CLAUDE.md it writes for you. Edit anything that doesn't sound like you.
- Test: ask Claude *"based on my CLAUDE.md, what are the most important things you know about me?"*

Done when: Claude's answer sounds like you.

8D Track

- Run `/personalize` too, same starting point
- Then go further: build the hierarchical CLAUDE.md cascade (global → project → worktree)
- Add a script that auto-updates your "Current Projects" section
- Architects: install the `obsidian-cairns-starter` and read the Cairns 5-doc guide

Done when: cascade loads correctly + Cairns starter is in place.

> Split room links posted in #agent-native-os and Zoom chat now. Rejoin at 11:00 AM.

LAYER 2

11:00 AM

MCPs + CLIs.

Your AI can see your world.

LAYER 2. CONCEPT

Your AI can see your world.

Without senses: Claude only knows what you type.

No inbox. No repos. No calendar. No Slack. Just the conversation. Every time.

With senses: Claude reads email, calendar, code, your notes, the web.

It can answer "what's on my plate today" with real data from your real world, without you copying anything in.

> **Two ways Claude reaches outside itself: MCPs and CLIs.**

THE TWO CHANNELS

MCPs, a polished door an app builds *for AI*. OAuth-based, structured. Examples: Gmail, Calendar, Notion, GitHub, Slack.

CLIs, a tool built *for humans* that Claude uses by typing commands. Examples: `gh` (GitHub), `op` (1Password), `vercel`, `supabase`.

WHAT YOU'RE WIRING TODAY

- **MCPs:** GitHub, Gmail, Calendar, Obsidian (filesystem)
- **CLIs:** `gh` for GitHub actions, `op` for secrets, `vercel` / `supabase` (sets up the capstone)
- **Knowledge:** Obsidian + Cairns vault. AI-readable transcripts, decisions, research

What you're building in L2

	4D	8D
MCPs	GitHub MCP. Gmail + Calendar via desktop connectors (OAuth, one click).	GitHub + 2 more (Slack, Notion, or filesystem). `/mcp` shows 3+ green servers.
CLIs	Verify `gh`, `op` (1Password), `vercel`, `supabase` all working. Run `which gh op vercel supabase`, all four return paths.	Same four CLIs, plus build comfort using them in Claude prompts (e.g., *"using `gh`, list every issue assigned to me across all my repos"*).
Secrets (everyone uses 1Password)	At least one secret in 1Password. At least one `op://` reference in your `settings.json`. Test that the MCP using it still works after restart.	All secrets in 1Password. Shell auto-loads `op://` env vars on every terminal session. Audit: `grep -r "sk-|ghp_|xoxb"`, zero results.
Knowledge	Point Claude at the Obsidian vault you set up in L1. Test Claude can read a real note.	Add Firecrawl + a custom CLI-backed Skill for a source the repo doesn't ship with. Pattern: Firecrawl → CLI → Skill.
Done when	Claude answers "what's on my calendar tomorrow?" using a connector AND "what are my open PRs?" using `gh`, same task, two channels.	3+ MCPs green, all four CLIs in active use, every secret in 1Password, zero plaintext keys on disk.

LAYER 2. DEMO

Wiring Claude's senses live

1. Tyler adds GitHub MCP, one config line. Asks: *"What are my open PRs?"*, agent fans out across all repos.
2. Claude Desktop, Connectors, Gmail OAuth, 22 seconds, green check. Claude can now read his inbox.
3. **CLI flip:** Tyler asks *"using `gh`, list every issue assigned to me across all my repos"*. Claude shells out to `gh issue list --assignee @me` and returns a clean list. Same task as the MCP, different channel.
4. Opens live AI Cairns vault: 7,300 transcripts, Supabase pgvector, Neo4j graph.
5. Query: *"What did Sara say about pricing in February?"*, returns actual quote with timestamp.

> Six months of meetings. Searchable in two seconds. No copy-paste. No searching through tabs. Just ask.

> **8D extension:** Tyler wires GitHub + 2 more MCPs, runs the audit (`grep -r "sk-\|ghp_\|xoxb" ~`, zero results), shows Neo4j graph traversal.

Now you build it

4D Track

- **MCPs:** wire GitHub MCP. Add Gmail + Calendar connectors via Claude desktop.
- **CLIs:** verify `gh auth status`. Ask Claude *"using `gh`, what are my open PRs?"*
- **Secrets:** store one secret in 1Password. Replace one plain-text value in `settings.json` with an `op://` reference. Restart Claude, verify it still works.

Done when: Claude answers a calendar question AND lists PRs via `gh` AND has at least one `op://` ref working.

8D Track

- **MCPs:** GitHub + 2 more (Slack/Notion/filesystem). `/mcp` shows 3+ green servers.
- **Secrets:** move *every* secret to `op://` refs. Wire shell auto-load so env vars resolve on every terminal session.
- **Audit:** `grep -r "sk-\|ghp_\|xoxb"`, zero results
- **Pattern:** Firecrawl → CLI → Skills, build a custom tool for a source the repo doesn't ship with.

Done when: 3 MCPs green, all secrets in 1Password, zero plaintext keys, one custom Skill working.

> Split room links in #agent-native-os. Rejoin at 12:00 PM.

BREAK

Lunch + Help Rooms

12:00 PM, 1:00 PM CT

4D Help Room

- Still stuck on CLAUDE.md not loading? Most common issue.
- MCP showing red? Second most common.
- Tell me where you got stuck, we fix it together.

Goal: everyone into Layer 3 on time.

8D Help Room

- Already wired up? Pre-build a 3rd MCP or explore the Cairns starter.
- Stuck on 1Password references? That's what I'm here for.

Back at 1:00 PM sharp.

Michael on Slack triage in #agent-native-os. Post any question there.

Before the Capstone you'll need Vercel and Supabase accounts. Links are in #agent-native-os, sign up during this break if you haven't yet. Takes about 5 minutes each.

LAYER 3

1:00 PM

Slash Commands.

Your AI takes action when you say a single word.

LAYER 3. CONCEPT

Your AI takes action when you say a single word.

Without triggers: you re-type the same prompt every time.

200-word morning prompt. Every day. Hoping you remembered everything. Hoping it comes out right.

With triggers: one word does all of it.

You write the prompt once. Give it a name. From that point on, you just type the name.

> **Think of a slash command as a saved workflow.** A named playbook with multiple steps that you trigger by typing one word. `/personalize` is a 7-step workflow. `/morning` is a multi-source briefing. Same playbook every time, customized to your inputs.

Plain analogy: a regular Claude prompt = asking a colleague to do something one time. A slash command = the saved playbook that colleague runs whenever you say the codename.

HOW IT WORKS

- A markdown file at `~/.claude/commands/name.md`
- The file IS the command. The filename IS the trigger.
- `$ARGUMENTS` lets it take free-form input: `/log-decision we're going with Mattermost`
- Commands can call other commands. Decisions compound. Second brain grows one file at a time.

> Every time you type the same thing to Claude more than twice, that is a slash command waiting to be written.

What you're building in L3

	4D	8D
Slash commands	Install and run one command from the repo (`/morning` or `/end-of-day`).	Write two from scratch. Second uses `$ARGUMENTS`. `/log-decision` wires into second brain.
Done when	One command runs and returns output.	`/log-decision` writes a file to `second-brain/decisions/` and you can open it.

LAYER 3. DEMO

/morning + /log-decision live

1. Tyler writes `/morning` from scratch, 3 lines, live on screen, no edits
2. Saves to `~/.claude/commands/morning.md`
3. Invokes `/morning`, reads calendar, Gmail, returns 3-bullet brief. Total time: 4 minutes.
4. Invokes `/log-decision we are going with Mattermost over Slack for internal comms`
5. Skill writes structured file with frontmatter to `~/Documents/second-brain/decisions/`
6. Tyler opens the file on screen

> That file is now searchable and provenance-tagged forever. Tyler just wrote a piece of his second brain in under 10 seconds.

> **The point:** writing a slash command is shorter than typing this paragraph.

LAYER 3. BUILD

Now you build it

4D Track

- Install `/morning` or `/end-of-day` from the repo
- Invoke it once

Done when: command runs and returns output.

8D Track

- Write `/end-of-day` from scratch
- Write `/log-decision` with `$ARGUMENTS`
- Invoke both

Done when: /log-decision writes a file to second-brain/decisions/

> Split room links in #agent-native-os. Rejoin at 2:00 PM.

LAYER 4

2:00 PM

Scheduling.

Your AI runs when you sleep.

LAYER 4. CONCEPT

Your AI runs when you sleep.

Without autonomy: your agent only works when you're watching.

Every morning brief requires you to open a terminal and type the command. Every digest requires you to remember to ask. The value stops the moment you stop.

With autonomy: the loop runs whether you're watching or not.

7am. Brief is already there. You didn't do anything. That is the point.

> Autonomy with a kill switch = operating asset.
> **Autonomy without a kill switch = liability.**

EVERY AUTONOMOUS LOOP HAS FOUR PARTS

- **Trigger**, a schedule (cron/launchd) or an event
- **Job**, a specific bounded task with a scope
- **Delivery**, where the output goes (Telegram, Obsidian, Slack)
- **Kill switch**, budget cap, time cap, retry cap, manual stop. Wired before you schedule. Not after.

> **The heartbeat pattern:** free local Ollama model decides WAKE or SLEEP every 5 minutes. Cloud model only fires on WAKE. Tyler's ran 8,640 times this month. Cost: $0.

What you're building in L4

	4D	8D
Scheduling	Documented plan: the file, the cron line, what it does, where output goes, kill switch command.	Real scheduled job. `/morning` at 7am tomorrow. Kill switch documented and tested.
Remote control	Watch the demo. Understand what native Remote Control means.	Start Remote Control, confirm the session is reachable from another device. Optional: one Channel plugin.
Done when	Plan is written and you could execute it tonight.	Job is scheduled, kill switch works, session reachable from another surface.

LAYER 4. DEMO

Heartbeat + Remote Control live

1. Tyler runs `tail -f ~/automations/heartbeat/logs/heartbeat.log`. WAKE/SLEEP decisions in real time
2. Five minutes ago: read Slack, task board, git status. Local Ollama model decided SLEEP. Cost: $0.
3. Tyler shows the next run happening live.
4. Tyler starts native Remote Control, opens same session from iPhone via Telegram
5. Sends "what is on my plate today" from phone. Mac Studio answers

> One session. Two surfaces. Phone sends the message, Mac Studio answers. That is what remote control means.

> **8D extension (Wade):** Langfuse at `langfuse.srv761271.hstgr.cloud`, filters by `agent_source=heartbeat`, drills into a trace, shows eval score, cost per call, secret redaction.

LAYER 4. BUILD

Now you build it

4D Track

- Look at the cron line for `/morning`
- Write your plan: "I will schedule /morning at 7am on [day]. Output to [location]. Kill switch: [command]."

Done when: plan is written and you could execute it tonight.

8D Track

- `crontab -e` or a launchd plist
- Schedule `/morning` for 7am tomorrow
- Document kill switch
- Start Remote Control, confirm reachable from another device
- Optional: one Channel plugin (Telegram, fakechat, or iMessage)

Done when: job scheduled, kill switch documented.

> Split room links in #agent-native-os. Rejoin at 3:30 PM, this is the merge into the capstone.

CAPSTONE

3:30 PM, everyone together

The Morning Brief System.

You wake up to a brief. It ran while you slept.

CAPSTONE

What /build-morning-brief does

1. Read blueprint → 2. Ask only blockers → 3. Config → 4. Compiler → 5. Delivery → 6. Manual test → 7. Schedule

- **Blockers it asks:** output surface, sources, delivery channel (Telegram bot default), auth, cost ceiling
- **Stack:** Vercel + Supabase + Claude Code. Telegram bot delivery. Obsidian markdown fallback.
- **Cost:** Vercel Hobby + Supabase free tier = $0

> Scheduling happens only **after** the manual test passes.

CAPSTONE. LIVE DEMO

Tyler types /build-morning-brief

```
/build-morning-brief gmail,calendar,slack,obsidian
```

WHAT TYLER SHOWS

1. Manual test runs. Brief generates.
2. Vercel preview URL, live web brief.
3. Telegram ping arrives on his phone.
4. Obsidian markdown fallback file on disk.
5. Supabase row appears.
6. Cron line scheduled. Kill switch documented.

WHAT YOU LEAVE WITH

4D

Vercel deploy + Obsidian markdown output. Telegram ping at 7am. Working URL to show your team Monday.

8D

Full Vercel + Supabase. Telegram + Slack + optional iMessage. 2-3 custom sources. launchd scheduled.

> That file did not exist 4 minutes ago. Tomorrow morning you will not type anything. It will already be there.

CAPSTONE. THE REVEAL

That is not a demo. That is a blueprint.

Source → Compile → Store → Ping → Inspect → Rerun

> This file did not exist 4 minutes ago.

After Sunday: run it once, then add one source that matters to your business.

CLOSING, 4:30 PM CT

Show us what you built

2-3 students. Screen share. 2-3 minutes each.

4D, 2 minutes

- ”Here is what I built.”
- ”Here is the one thing I will use Monday.”

Low pressure. No screen-share required.

8D, 3 minutes

- Screen-share required
- Show the artifact, live URL, terminal, or Telegram ping
- ”Here is what I will add next.”

Screen-share required.

> Want to share? Post in #agent-native-os now. Sara will call on you.

Tyler steps out for 30 seconds.

His phone rings.

Caller ID: Gigawatt.

The agent calls Tyler, asks a question about the workshop, updates a Notion page live.

> This is not curriculum. This is what the architecture becomes when you keep building.

CLOSING

If today moved too fast, that is fine.

If you're in the 4D track and enrolled in Summer Camp, your next step is the 8D session on May 31.

We want you at 60D level before camp starts. Your homework between now and then: live in Claude Code. Open the repo Monday. Tell Claude to walk you through the next thing. The second pass is when it clicks.

> **The May 31 retake is free.** No application. Just show up, and show up at 8D level.

"If today moved too slow and you want to go deeper, the Lab is the place."

Weekly office hours. Live builds. Bring your own work. We build it with you.

> **Summer Camp** is the 90-day intensive. $950, includes 90 days of Lab. Link in #agent-native-os after close.

CLOSING

The Lab + Summer Camp

The Lab

- **$99** founding member (first 100)
- $149 regular after that
- Weekly office hours
- Blueprint library + Cairns alpha
- Retake the workshop free, any month
- aibuildlab.com/lab

Summer Camp

- **$950**, includes 90 days Lab
- 90-day intensive: June through August
- Ship production agents every week
- Same stack you installed today
- Second session (May 31 minimum) before applying
- aibuildlab.com/summer-camp

> Founding pricing closes at 100. Link dropping in #agent-native-os now.

CLOSING

What you did not see today.

"You bought THIS."

The ecosystem graph toggles to ~250 nodes. Every dim node lights up.

> You did not see: compound engineering, 12-Factor Agents, EXO multi-machine clustering, agent-native CRM, Cairns librarian agent at full power.

> All of it is in the repo. Your Claude knows where it is.

Open Claude Code Monday morning, in this repo, and say:

> "I just finished the workshop. Walk me through the next thing."

Your Claude knows what to do.

Office hours Friday 11am CDT. See you in #agent-native-os.

Repo is the school. Your Claude is the teacher. We are the TAs in the hallway.

Now go build.

#agent-native-os · Office hours Friday 11am CDT · Thanks for showing up.